



CENTRO DE CIENCIAS DE LA INGENIERIA

CARRERA: INGENIERIA EN ROBOTICA

EXAMEN SEGUNDO PARCIAL

INTEGRANTES:

LEONARDO DE LIRA ARTEAGA

ID: 261555

BRIAN LEYVA DE LA TORRE

ID: 333744

RAFAEL MEZA LOPEZ

ID: 233765

RAFAEL WALDO JONGUITUD

ID: 265727



ASIGNATURA: VISION ROBOTICA

DOCENTE: CARLOS ALBERTO LOPEZ
HERNANDEZ

GRADO Y GRUPO: 8vo "A"

FECHA DE ENTREGA: 11-05-2026

ABSTRACT

This project presents the development and implementation of a custom Python library focused on digital image processing and computer vision. The primary goal is the detection, segmentation, and counting of objects in overhead images with sufficient contrast. The system employs a pipeline consisting of preprocessing via Gaussian blur and contrast enhancement, segmentation thresholding, and morphological filtering. To address the challenge of overlapping or touching objects, the algorithm integrates distance transform, segmentation technique. Consequently, the system provides not only an automated count but also extracts critical geometric features such as area, perimeter, centroids, and corner detection. Experimental results confirm high accuracy in controlled environments, demonstrating the robustness of classical vision techniques for automated inspection and data extraction tasks.

INTRODUCCION

La visión artificial es clave para automatizar la inspección de objetos y reducir errores humanos en procesos de análisis visual. Este proyecto presenta un sistema desarrollado en Python diseñado para detectar, contar y medir objetos automáticamente a partir de imágenes digitales.

El software utiliza un flujo de trabajo que incluye el realce de contraste, la segmentación y un algoritmo para separar objetos que se tocan entre sí. Más allá del simple conteo, el sistema extrae datos técnicos precisos como el área, el perímetro y la ubicación exacta de cada elemento, demostrando ser una solución eficiente para tareas de control de calidad y análisis geométrico.

DEFINICION DEL PROBLEMA Y APLICACIÓN PRACTICA

Este proyecto resuelve la ineficiencia y el error humano propios del conteo manual mediante un sistema de visión computacional. El software automatiza la identificación y el inventariado de piezas en imágenes superiores, logrando aislar cada unidad del fondo con precisión.

El propósito central es obtener un registro exacto del número de elementos y extraer sus métricas morfológicas (superficie, contorno y ubicación). Esta solución optimiza procesos de control de calidad, clasificación de productos y monitoreo industrial, transformando una tarea visual compleja en un flujo de datos preciso y automatizado.

MARCO TEORICO

- **Visión Artificial:** Análisis automatizado de imágenes mediante el procesamiento de matrices de píxeles en Python.
- **Filtro Gaussiano:** Reducción de ruido y suavizado de imagen para mejorar la precisión de detección.
- **Umbralización:** Binarización automática (blanco y negro) que separa eficientemente el objeto del fondo.
- **Algoritmo Watershed:** Técnica morfológica que permite dividir y contabilizar objetos que están en contacto físico.
- **Momentos Espaciales:** Análisis matemático para extraer métricas geométricas: área, perímetro y centroide.

METODOLOGIA

Acondicionamiento: Conversión a escala de grises, suavizado con Filtro Gaussiano y realce de contraste.

Binarización: Aplicación de Umbral de Otsu y limpieza morfológica para aislar la silueta de los objetos.

Desacoplamiento: Uso de Transformada de Distancia y Watershed para dividir piezas que se tocan entre sí.

Análisis Geométrico: Localización de contornos y cálculo de métricas (área, perímetro y centroide) mediante momentos.

Reporte: Etiquetado visual sobre la imagen y exportación de datos cuantitativos finales.

RESULTADOS

“EXPLICACION DEL PRE-PROCESAMIENTO”

La primera etapa del sistema corresponde al pre procesamiento de la imagen, cuyo propósito es mejorar la calidad visual antes de realizar la segmentación.

Inicialmente, la imagen RGB es convertida a escala de grises para reducir la complejidad computacional y trabajar únicamente con niveles de intensidad. Posteriormente, se aplica un filtrado espacial mediante un filtro Gaussiano, el cual reduce ruido y suaviza pequeñas variaciones de intensidad presentes en la imagen.

Finalmente, se implementa una transformación de intensidad utilizando ajuste de brillo y contraste, con el objetivo de mejorar la diferenciación entre objetos y fondo, facilitando así la segmentación posterior.

“DESCRIPCION DE LA SEGMENTACION Y EL CONTEO”

La segmentación se realiza mediante un proceso de umbralización automática utilizando el método de Otsu, el cual calcula un valor de umbral óptimo para separar regiones correspondientes a objetos del fondo de manera automática.

Posteriormente, se aplican operaciones morfológicas de apertura y cierre para eliminar ruido, rellenar pequeñas discontinuidades y mejorar la forma de las regiones segmentadas.

Para la detección de objetos pegados o muy cercanos, se emplea el algoritmo Watershed, el cual permite dividir regiones conectadas en objetos individuales mediante transformada de distancia y componentes conectados.

Adicionalmente, el sistema incorpora detección de bordes utilizando el algoritmo Canny y detección de esquinas mediante Harris, permitiendo obtener información geométrica adicional de las piezas analizadas.

Finalmente, el conteo se realiza mediante el análisis de contornos. Cada objeto identificado es etiquetado individualmente y se calculan propiedades geométricas básicas, incluyendo área, perímetro y coordenadas del centroide.

“LIMITACIONES POR OBJETOS PEGADOS, SOMBRAS O PRESPECTIVA”

Durante el desarrollo se identificaron ciertas limitaciones del sistema. Cuando los objetos se encuentran muy pegados entre sí, puede presentarse unión de regiones, dificultando la separación individual incluso utilizando Watershed.

Las sombras y cambios bruscos de iluminación también pueden afectar la umbralización, generando errores de segmentación o alterando el conteo final. De igual manera, el método presenta mejor desempeño en imágenes cenitales con perspectiva controlada; inclinaciones

de cámara o deformaciones geométricas pueden modificar la forma aparente de los objetos y afectar la precisión del análisis.

En general, el sistema funciona de manera más estable en escenarios con iluminación uniforme y contraste suficiente entre objeto y fondo.

¿QUE SE LOGRO?

El sistema desarrollado logró segmentar y contar correctamente objetos presentes sobre una superficie de trabajo en imágenes cenitales de prueba. Se obtuvieron resultados satisfactorios en escenarios donde existe suficiente contraste entre los objetos y el fondo.

La librería permitió visualizar cada objeto identificado mediante contornos y etiquetas numéricas, además de generar métricas geométricas útiles para el análisis individual de cada pieza.

Asimismo, se verificó el funcionamiento de los módulos de pre-procesamiento, segmentación, detección de bordes y esquinas, cumpliendo con los requerimientos técnicos establecidos para el proyecto.

“CODIGO LIBRERÍA”

--- LIBRERÍA ---

#El sistema fue diseñado para imágenes cenitales con contraste suficiente entre objeto y fondo.

#Se empleó umbralización global con Otsu, por lo que el desempeño es óptimo cuando existe separación de intensidad.

#En escenarios con iluminación variable o colores similares, podría mejorarse usando umbralización adaptativa o segmentación basada en color.

```
codigo_libreria = ""
```

```
import cv2
```

```
import numpy as np
```

```
class ContadorObjetos:
```

```
def __init__(self):
```

```
    pass
```

```
# PREPROCESAMIENTO
```

```
def mejorar_imagen(self,imagen):
```

```
    imagen_gris =  
    cv2.cvtColor(imagen,cv2.COLOR_BGR2GRAY)
```

```
# Suavizar ruido
```

```
    imagen_suave =  
    cv2.GaussianBlur(imagen_gris,(5,5),0)
```

```
# Mejorar contraste
```

```
    imagen_mejorada =  
    (cv2.convertScaleAbs(imagen_suave,alpha  
    =1.15,beta=5))
```

```
    return imagen_mejorada
```

```
# SEGMENTACIÓN
```

```
def separar_objetos(self,imagen_gris):
```

```
# Threshold automático
```

```
    _, mascara =  
    cv2.threshold(imagen_gris,0,255,cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
```

```
    kernel = np.ones((3,3),np.uint8)
```

```
# Limpiar ruido
```

```
    mascara =  
    cv2.morphologyEx(mascara,cv2.MORPH_OPEN,kernel,iterations=1)
```

```

# Cerrar huecos

mascara =
cv2.morphologyEx(mascara,cv2.MORPH_C
LOSE,kernel,iterations=2)

return mascara

# BORDES

def sacar_bordes(self,imagen_gris):

    bordes =
cv2.Canny(imagen_gris,50,150)

    return bordes

# ESQUINAS

def
sacar_esquinas(self,imagen_gris,mascara):

    solo_objetos =
(cv2.bitwise_and(imagen_gris,imagen_gris
,mask=mascara))

    esquinas =
cv2.cornerHarris(np.float32(solo_objetos),
2,3,0.04)

    return esquinas

# CONTEO

def
contar_objetos(self,mascara,imagen_origi
nal):

    imagen_resultado =
(imagen_original.copy())

    datos_objetos = []

    # Watershed para separar objetos
pegados

    kernel = np.ones((3,3),np.uint8)

    fondo_seguro =
cv2.dilate(mascara,kernel,iterations=2)

```

```

    distancia =
(cv2.distanceTransform(mascara,cv2.DIST
_L2,5))

    _, primer_plano =
(cv2.threshold(distancia,0.25 *
distancia.max(),255,0))

    primer_plano =
np.uint8(primer_plano)

    zona_desconocida =
cv2.subtract(fondo_seguro,primer_plano)

    _, marcadores =
(cv2.connectedComponents(primer_plano
))

    marcadores = (marcadores + 1)

    marcadores[zona_desconocida ==
255] = 0

    marcadores =
cv2.watershed(imagen_original,marcador
es)

    numero_objeto = 1

    for etiqueta in
np.unique(marcadores):

        if etiqueta <= 1:

            continue

            mascara_objeto =
np.zeros(mascara.shape,dtype="uint8")

            mascara_objeto[marcadores ==
etiqueta] = 255

            contornos, _ =
(cv2.findContours(mascara_objeto,cv2.RE
TR_EXTERNAL,cv2.CHAIN_APPROX_SIMPL
E))

            if len(contornos) == 0:

                continue

```

```

        contorno =
max(contornos,key=cv2.contourArea)

        area = cv2.contourArea(contorno)

        # eliminar ruido

        if area < 300:

            continue

        x, y, ancho, alto =
(cv2.boundingRect(contorno))

        perimetro =
(cv2.arcLength(contorno,True))

        momentos =
(cv2.moments(contorno))

        if momentos["m00"] != 0:

            centro_x =
int(momentos["m10"]/momentos["m00"]
)

            centro_y =
int(momentos["m01"]/momentos["m00"]
)

        else:

            centro_x = 0

            centro_y = 0

cv2.drawContours(imagen_resultado,[con
torno],-1,(0,255,0),2)

cv2.circle(imagen_resultado,(centro_x,cen
tro_y),5,(255,0,0),-1)

cv2.putText(imagen_resultado,f"#{numer
o_objeto}",(x, y -
10),cv2.FONT_HERSHEY_SIMPLEX,0.6,(0,0,
255),2)

        datos_objetos.append({

```

```

        "numero":numero_objeto,

        "area":area,

        "perimetro":perimetro,

        "centro":(centro_x,centro_y)

    })

    numero_objeto += 1

    return
(imagen_resultado,datos_objetos)

"""

with
open("vision_lib.py","w",encoding="utf-
8") as archivo:

    archivo.write(codigo_libreria)

print("Librería guardada correctamente")

```

"CODIGO DE PRUEBA"

```

# --- PRUEBA LIBRERIA ---

import matplotlib.pyplot as plt
from vision_lib import
ContadorObjetos
import cv2
import os

nombre_imagen = "mas2.jpg"

if not
os.path.exists(nombre_imagen):

    print("No se encontró la
imagen")

else:

    foto =
cv2.imread(nombre_imagen)

    contador =
(ContadorObjetos())

# PREPROCESAMIENTO

```

```

    imagen_lista =
(contador.mejorar_imagen(foto))

    # SEGMENTACIÓN
    mascara_final =
(contador.separar_objetos(imagen
_lista))

    # BORDES
    bordes =
(contador.sacar_bordes(imagen_li
sta))

    # ESQUINAS
    esquinas =
(contador.sacar_esquinas(imagen_
lista,mascara_final))

    imagen_esquinas =
(foto.copy())

    limite = (0.05 *
esquinas.max())

    imagen_esquinas[esquinas >
limite] = [255, 0, 0]

    # CONTEO
    imagen_final, datos =
(contador.contar_objetos(mascara
_final,foto))

    # MOSTRAR RESULTADOS
    plt.figure(figsize=(18, 10))

    plt.subplot(2,3,1)
    plt.title("Imagen Original")

    plt.imshow(cv2.cvtColor(foto,cv2
.COLOR_BGR2RGB))
    plt.axis("off")

    plt.subplot(2,3,2)

plt.title("Preprocesamiento")

plt.imshow(imagen_lista,cmap="gr
ay")
    plt.axis("off")

    plt.subplot(2,3,3)
    plt.title("Segmentación")

    plt.imshow(mascara_final,cmap="g
ray")
    plt.axis("off")

    plt.subplot(2,3,4)
    plt.title("Detección de
Bordes")

```

```

plt.imshow(bordes,cmap="gray")
    plt.axis("off")

    plt.subplot(2,3,5)
    plt.title("Detección de
Esquinas")

    plt.imshow(cv2.cvtColor(imagen_e
squinas,cv2.COLOR_BGR2RGB))
    plt.axis("off")

    plt.subplot(2,3,6)
    plt.title("Resultado Final")

    plt.imshow(cv2.cvtColor(imagen_f
inal,cv2.COLOR_BGR2RGB))
    plt.axis("off")

    plt.tight_layout()
    plt.show()

    # Resultados
    print("-" * 40)
    print("RESULTADOS")
    print("-" * 40)
    print("Objetos
encontrados:",len(datos))
    print("-" * 40)

    for objeto in datos:

        print(f"Objeto
#{objeto['numero']}")
        print(f"Área: "
f"{objeto['area']:.2f}")
        print(f"Perímetro: "
f"{objeto['perimetro']:.2f}")
        print(f"Centro: "
f"{objeto['centro']}")
        print("-" * 20)

```

"RESUTADOS DE CODIGO PRUEBA"



Preprocesamiento



Segmentación



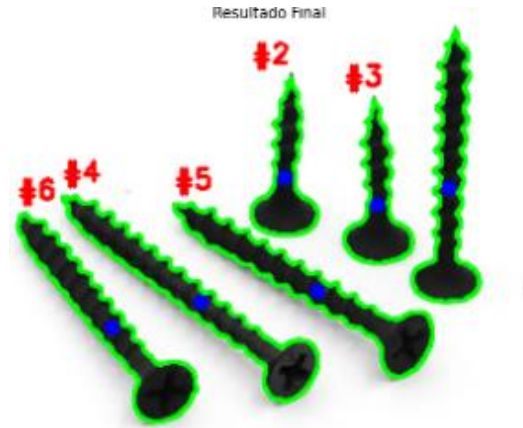
Detección de Bordos



Detección de Esquinas



Resultado Final



RESULTADOS

Objetos encontrados: 6

Objeto #1
Área: 3307.50
Perímetro: 496.80
Centro: (295, 109)

Objeto #2
Área: 1975.50
Perímetro: 283.52
Centro: (185, 102)

Objeto #3
Área: 2059.50
Perímetro: 301.32
Centro: (247, 120)

Objeto #4
Área: 4079.00
Perímetro: 539.33
Centro: (129, 185)

Objeto #5
Área: 3899.00
Perímetro: 513.39
Centro: (207, 178)

Objeto #6
Área: 3695.00
Perímetro: 440.42
Centro: (69, 202)

"LINK GITHUB"

<https://github.com/LeonardoDLA/Segmentacion-de-objetos>

"COSAS QUE SE COMPLICARON Y SUS SOLUCION"

Problemas encontrados durante el desarrollo

1. Unión de objetos cercanos

En etapas iniciales, varios objetos aparecían unidos como una sola región debido a operaciones morfológicas excesivas y a la cercanía física entre piezas.

***Solución aplicada: ***

Se ajustaron los parámetros morfológicos y se implementó segmentación mediante *Watershed* para mejorar la separación de objetos pegados.

2. Conteo incorrecto de objetos

Inicialmente se detectaban regiones pequeñas de ruido como objetos reales, generando conteos erróneos.

***Solución aplicada: ***

Se agregó un filtro por *área mínima*, eliminando regiones pequeñas no representativas.

3. Dependencia del contraste objeto-fondo

Se observó que el sistema funciona mejor cuando existe suficiente contraste entre el objeto y el fondo, especialmente con objetos oscuros sobre superficies claras.

***Solución aplicada: ***

Se mejoró el pre procesamiento mediante suavizado y ajuste de intensidad para aumentar la diferenciación visual.

4. Sensibilidad a iluminación y sombras

Cambios de iluminación o sombras afectaban la segmentación binaria y modificaban el conteo.

***Solución aplicada: ***

Se utilizó *thresholding automático (Otsu)* para adaptar el umbral de segmentación a cada imagen.

5. Separación incompleta de bordes

En algunos casos los contornos aparecían incompletos, dificultando el análisis geométrico.

***Solución aplicada: ***

Se incorporó *detección de bordes con Canny* y operaciones morfológicas de cierre para completar regiones.

Limitación general del sistema

El método presenta mejor desempeño en imágenes cenitales con fondo relativamente uniforme y contraste suficiente entre objeto y superficie. Objetos demasiado juntos, sombras fuertes o cambios bruscos de iluminación pueden afectar el desempeño.

CONCLUSION

El proyecto demuestra la viabilidad de automatizar el análisis de objetos mediante visión artificial, logrando integrar con éxito el pre procesamiento, la segmentación y la extracción de métricas (área y perímetro). El uso de algoritmos como Watershed permitió resolver de manera eficiente el conteo de objetos cercanos, demostrando que la combinación de técnicas de procesamiento digital es una herramienta robusta para el control de inventarios o análisis de formas.

BIBLIOGRFIAS

- Bradski, G., y Kaehler, A. (2008). Learning OpenCV: Computer Vision with the OpenCV Library. O'Reilly Media.
- González, R. C., y Woods, R. E. (2018). Digital Image Processing (4ta ed.). Pearson.
- Szeliski, R. (2022). Computer Vision: Algorithms and Applications (2da ed.). Springer Nature.
- Otsu, N. (1979). A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1), 62-66.
- Rosebrock, A. (2016). *Deep Learning for Computer Vision with Python*. PyImageSearch.